

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional La Plata

Ingeniería en Sistemas de Información

Administración de Sistemas de Información

TRABAJO PRÁCTICO INTEGRADOR

Bases de datos vectoriales

Fundamentos, arquitectura y aplicación
en un buscador de jugadores de fútbol

Grupo N.º: _____

Apellido y nombre	Correo electrónico

Fecha de presentación: _____ / _____ / 2026

Universidad Tecnológica Nacional, Facultad Regional La Plata
Buenos Aires, Argentina · Año 2026

ÍNDICE

Resumen y palabras clave	3
1. Introducción	4
2. Descripción del tema	5
3. Características	6
4. Desarrollo	9
5. Ambientes de desarrollo y aplicaciones	11
6. Casos de estudio	13
7. Bases vectoriales aplicadas a RAG	14
8. Caso práctico: buscador de jugadores	15
8.1 Problema y fuente	15
8.2 Stack implementado	16
8.3-8.4 Representación y ranking	17
8.5-8.6 Implementación y generación	18
8.7-8.8 Demostración	19
8.9 Resultados comprobados	20
8.10-8.11 Pruebas y evaluación	21
9. Análisis crítico y administración	22
10. Conclusión	23
Referencias y materiales	24

Resumen

Las bases de datos vectoriales permiten almacenar y recuperar representaciones numéricas de textos, imágenes y otros contenidos mediante criterios de similitud. Su importancia para los sistemas de información reside en complementar las consultas exactas con mecanismos capaces de relacionar expresiones diferentes de una misma necesidad. El resultado depende tanto de la representación elegida como del índice, los filtros y la calidad de los datos.

Este trabajo presenta los conceptos de vector, embedding, distancia y búsqueda de vecinos próximos; describe la arquitectura de una solución vectorial y analiza sus aplicaciones, limitaciones y necesidades de administración. También explica su participación en Retrieval-Augmented Generation (RAG), una arquitectura que incorpora información recuperada al contexto de un modelo generativo. Se distingue la base vectorial, como infraestructura de recuperación, del sistema completo que produce una respuesta.

El caso práctico consiste en un buscador de jugadores de fútbol desarrollado en Python y Streamlit. Utiliza Chroma con embeddings locales MiniLM de 384 dimensiones, filtros por metadatos y reordenamiento por atributos. El corpus principal contiene perfiles derivados de tres CSV de FIFA/EA FC, correspondientes a una selección de edición y actualización determinada. Sus puntuaciones pertenecen al videojuego; no representan estadísticas oficiales de partidos ni información deportiva actualizada.

La verificación combina la ejecución de 35 pruebas automatizadas con una consulta real sobre la colección completa. Los resultados permiten explicar la diferencia entre cercanía semántica, cumplimiento de filtros y ordenamiento numérico. La generación de informes con un LLM es opcional y se separa explícitamente de la recuperación local. Se presentan los alcances de la evidencia obtenida y una propuesta de evaluación futura de relevancia.

Palabras clave

Bases de datos vectoriales, embeddings, similitud coseno, HNSW, búsqueda semántica, RAG, Chroma, scouting de fútbol.

1. Introducción

1.1 Contexto y problema

Las organizaciones administran datos estructurados y contenido de formato heterogéneo: documentos, consultas de clientes, descripciones de productos, imágenes y registros de texto libre. Almacenar esa información no garantiza que pueda recuperarse de manera útil. Quien consulta puede desconocer la terminología del documento o formular una necesidad conceptual que no coincide con sus palabras.

La recuperación léxica emplea términos e índices invertidos. Es particularmente valiosa para nombres, códigos y expresiones exactas. Sin expansión de consultas, una búsqueda literal por «automóvil» puede omitir un texto que sólo diga «coche». El desajuste de vocabulario motiva el uso de representaciones que capturan relaciones de significado. Los métodos clásicos también admiten mejoras, como sinónimos y realimentación de relevancia; por eso no corresponde reducir toda recuperación tradicional a una simple igualdad de cadenas (Manning et al., 2008).

En el ámbito deportivo, una persona puede pedir «un delantero con regate y desborde» sin conocer los jugadores disponibles ni los nombres de todas las columnas del conjunto de datos. Esa consulta combina una intención expresada en lenguaje natural, una posición y atributos cuantificables. El caso permite estudiar cuándo conviene emplear similitud vectorial y cuándo resulta necesario aplicar restricciones y ordenamientos exactos.

1.2 Objetivos y metodología

El objetivo general es comprender el funcionamiento de las bases vectoriales y evaluar su incorporación a un sistema de información. Los objetivos específicos son explicar los fundamentos, comparar alternativas de implementación, analizar casos publicados y demostrar una aplicación funcional con evidencia trazable.

La metodología combina revisión bibliográfica, análisis del código del proyecto, comprobación del corpus y ejecución de pruebas. Las afirmaciones sobre el prototipo se sustentan en sus archivos y resultados locales. Los casos empresariales se presentan como experiencias publicadas por sus autores, sin extrapolar sus métricas al buscador desarrollado.

2. Descripción del tema

2.1 Vector, embedding y espacio de representación

Un vector es una secuencia ordenada de números: $x = (x_1, x_2, \dots, x_n)$. La cantidad n define su dimensionalidad. En una representación numérica diseñada manualmente, las componentes podrían ser edad, velocidad y regate. En un embedding de texto aprendido, las dimensiones generalmente no corresponden a atributos humanos independientes e interpretables.

Un embedding es una representación obtenida mediante un modelo que transforma el contenido en un vector. La representación conserva relaciones útiles para una tarea, pero no constituye una comprensión infalible del significado. Textos relacionados pueden ocupar posiciones próximas y textos diferentes pueden recibir representaciones similares. Sentence-BERT mostró una manera eficiente de producir representaciones de oraciones comparables mediante similitud coseno (Reimers y Gurevych, 2019).

El espacio de representación depende del modelo. Dos vectores con igual cantidad de componentes no son necesariamente comparables si se generaron con modelos distintos. Para consultar una colección deben mantenerse compatibles el modelo de documentos y consultas, el preprocesamiento y la métrica. Cambiar el modelo suele requerir una nueva indexación.

2.2 Qué almacena una base vectorial

La unidad lógica reúne un identificador, un vector y metadatos; también puede guardar el contenido original o una referencia. En el buscador, el documento conserva la ficha del jugador, el embedding representa su perfil y los metadatos incluyen identidad, edad, posiciones y atributos. Esas partes cumplen funciones diferentes: recuperar por semejanza, filtrar y mostrar evidencia.

Una base vectorial agrega gestión de colecciones, persistencia y operaciones de consulta al cálculo de vecinos. Las capacidades de concurrencia, replicación y seguridad varían entre productos y modalidades de despliegue. Una biblioteca de índices y un sistema gestor completo no ofrecen necesariamente las mismas garantías.

La búsqueda semántica ordena candidatos según una representación. No demuestra por sí sola que el primer resultado satisfaga todos los requisitos ni sustituye la validación de los datos de origen.

3. Características: similitud y restricciones

3.1 Métricas de comparación

La similitud coseno compara la orientación de dos vectores no nulos. Su definición es $\cos(a,b) = (a \cdot b) / (||a|| \ ||b||)$. Sus valores teóricos están entre -1 y 1. La distancia coseno suele definirse como $d = 1 - \cos(a,b)$: un valor menor expresa mayor cercanía. La similitud no es una probabilidad de respuesta correcta.

La distancia euclídea mide la separación entre puntos: $d(a,b) = \sqrt{\sum_i (a_i - b_i)^2}$. El producto interno suma los productos de las componentes y también depende de sus magnitudes. Si todos los vectores están normalizados a norma uno, ordenar por producto interno, coseno o distancia euclídea produce relaciones equivalentes de cercanía.

Concepto	Uso e interpretación
Distancia coseno	Menor distancia implica mayor cercanía angular.
Score = 1 - distancia	Recupera la similitud coseno; puede ser negativa.
Filtro de metadatos	Exige una condición sobre un atributo del registro.
Orden por atributos	Prioriza valores numéricos dentro de los candidatos.

Tabla 1. Operaciones distintas que intervienen en el buscador. Elaboración propia.

3.2 Filtros y búsqueda híbrida

Una edad máxima o una liga se deben expresar como condiciones estructuradas cuando sean requisitos obligatorios. Escribir «menor de 25 años» en una consulta no garantiza su cumplimiento mediante embeddings. En el prototipo, el usuario configura los filtros; el texto libre no los crea automáticamente.

Combinar similitud y filtros integra información estructurada con recuperación vectorial. En otros contextos, «búsqueda híbrida» designa específicamente la combinación de señales léxicas y densas, con fusión de rankings. Conviene aclarar cuál de estas operaciones se implementa: el comparador LIKE del proyecto es una rama de comparación, no un ranking fusionado con Chroma.

3. Características: vecinos e indexación

3.3 Búsqueda exacta y aproximada

La búsqueda exacta compara la consulta contra todos los vectores elegibles y selecciona los más cercanos. Su costo de comparación crece aproximadamente con $N \cdot D$, donde N es la cantidad de registros y D la dimensión. Puede ser una opción adecuada para colecciones pequeñas o para construir una referencia experimental.

La búsqueda aproximada de vecinos próximos (ANN) reduce el trabajo explorando sólo una parte del espacio. A cambio, puede omitir elementos del top-k exacto. Ese compromiso se controla con la elección y configuración del índice. Un vecino correcto desde el punto de vista geométrico tampoco equivale necesariamente a un resultado relevante para una persona.

3.4 HNSW

HNSW organiza los vectores en un grafo con varios niveles. Los niveles superiores permiten desplazamientos amplios y los inferiores refinan la búsqueda. La consulta parte de un punto de entrada y explora conexiones que acercan al vector buscado. Se trata de una estructura de navegación, no de un árbol ordenado por una única clave (Malkov y Yashunin, 2020).

Entre sus parámetros se encuentran la conectividad del grafo y la amplitud de exploración durante construcción y consulta. Aumentar esa exploración puede mejorar la recuperación de vecinos a costa de memoria o tiempo. No existe una configuración óptima universal: depende del corpus, los filtros y la exigencia de recall del índice.

3.5 Otras estrategias

Los índices IVF agrupan el espacio y consultan algunas particiones. La cuantización de productos aproxima los vectores mediante códigos compactos para reducir almacenamiento y acelerar comparaciones. Estas estrategias admiten combinaciones y requieren medir su pérdida de exactitud. Un índice plano conserva utilidad como referencia y como alternativa para escalas acotadas.

En el proyecto, Chroma utiliza HNSW con distancia coseno y puede mantener también un buffer de fuerza bruta. Por eso el tiempo de una llamada a su API no se debe presentar como tiempo exclusivo de navegación por el grafo.

3. Características: persistencia y escalabilidad

3.6 Dimensionamiento

El almacenamiento mínimo de vectores depende de su cantidad, dimensión y precisión. Con float32 se requieren $N \times D \times 4$ bytes. Para un millón de vectores de 384 dimensiones, el resultado es 1.536.000.000 bytes, aproximadamente 1,54 GB decimales. Esta cuenta excluye grafo, documentos, metadatos, índices auxiliares y sobrecarga del gestor.

En el corpus del buscador, 20.421 vectores de 384 dimensiones equivalen a 31.366.656 bytes de componentes float32, aproximadamente 31,37 MB. No es una medición del tamaño de la base ni del consumo del proceso: cargar MiniLM, estructuras de Python y datos de consulta agrega memoria.

3.7 Persistencia y ciclo de vida

La persistencia permite reutilizar la colección entre ejecuciones. La aplicación debe relacionar los vectores con una versión de los datos y del perfil textual. Si cambia la fuente y se consulta un índice viejo, los resultados pueden parecer válidos pero corresponder a otra versión. Identificadores estables y huellas del corpus ayudan a detectar esa situación.

Actualizar o eliminar registros requiere conocer el comportamiento del motor. Algunos índices marcan elementos y difieren tareas de mantenimiento. Las copias de seguridad y la recuperación deben abarcar datos, configuración y modelo, además de verificar que la restauración conserve las asociaciones entre vectores y fuentes.

3.8 Distribución y disponibilidad

El particionamiento reparte los registros entre nodos; la replicación mantiene copias para tolerar fallos o atender consultas. Ambas funciones agregan coordinación y decisiones sobre consistencia. No toda base vectorial local proporciona distribución ni alta disponibilidad por el solo hecho de almacenar embeddings.

El prototipo utiliza persistencia local. No demuestra escalabilidad horizontal, operación multiusuario ni tolerancia a la caída del equipo. Para incorporarlo a una organización habría que definir concurrencia, respaldos, recuperación, permisos y monitoreo, y probarlos con una carga representativa.

4. Desarrollo: arquitectura e ingesta

4.1 Separación de procesos

Una solución vectorial distingue la preparación de la colección y la atención de consultas. La primera puede ejecutarse por lotes; la segunda requiere una latencia compatible con el uso interactivo. Separarlas evita volver a vectorizar todo el corpus por cada búsqueda y permite controlar cuándo se incorporan cambios.

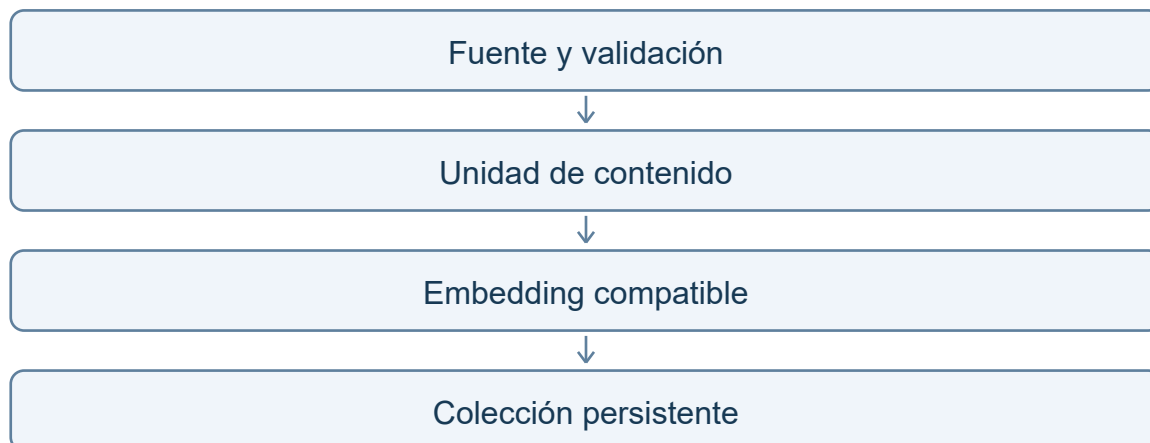


Figura 1. Flujo general de indexación. Elaboración propia.

4.2 Preparación y fragmentación

En documentos extensos suele ser necesario dividir el texto en fragmentos. Un fragmento demasiado pequeño pierde contexto; uno muy largo puede mezclar temas y superar la longitud admitida por el modelo. El solapamiento reduce cortes bruscos, pero duplica contenido y puede producir resultados redundantes. Tamaño y solapamiento deben validarse sobre los documentos reales.

En el caso de fútbol, la fuente principal es tabular: se construye una ficha por entidad dentro del snapshot seleccionado. No se implementa extracción de PDF por páginas ni fragmentación de reglamentos. El perfil breve que se vectoriza se separa del documento detallado que se muestra y eventualmente se envía al generador.

4.3 Metadatos y trazabilidad

Los metadatos deben conservar la identidad y la procedencia necesarias para interpretar un resultado: tipo de entidad, versión de los datos, fecha y vínculo de origen. En colecciones con información restringida también deben acompañar las condiciones de acceso.

La ingesta incluye controles de tipos, duplicación y vínculos entre entidades. Un identificador estable permite reconocer registros, pero no garantiza por sí solo que una actualización elimine datos obsoletos. El procedimiento debe definir expresamente cómo se reemplaza o reconstruye la colección cuando el contenido cambia.

4. Desarrollo: consulta y rendimiento

4.4 Proceso de recuperación

Una consulta se valida, se transforma con el modelo de embeddings y se combina con los filtros. El motor devuelve identificadores, documentos, metadatos y distancias. A continuación puede aplicarse reordenamiento y seleccionar el top-k que se mostrará o incorporará a un prompt. La aplicación debe preservar la distinción entre esos órdenes.

En el buscador, la llamada medida a Chroma recibe un vector ya calculado. Esto permite separar el tiempo del embedding del tiempo de `collection.query`. La segunda medición incluye trabajo del cliente, filtros y materialización de documentos: no es tiempo puro del índice. La métrica de recuperación termina antes del reordenamiento por atributos y de la generación.

4.5 Factores que afectan la latencia

El tamaño del corpus, la dimensión, la cantidad de candidatos, la selectividad del filtro, el estado de caché y el volumen de datos retornado influyen en la consulta. Recuperar miles de perfiles para reordenarlos puede costar más que pedir cinco vecinos. También importan el dispositivo y la actividad simultánea de otros procesos.

Una evaluación de rendimiento debe registrar configuración, hardware y carga; separar arranque frío y consultas posteriores; repetir consultas; e informar distribuciones, por ejemplo mediana y percentil 95. Una única medición describe una ejecución, pero no justifica una promesa de tiempo de respuesta ni una comparación general entre tecnologías.

4.6 Dimensionalidad y calidad

Más dimensiones no implican automáticamente mejor recuperación. El modelo y sus datos de entrenamiento determinan qué relaciones representa. En alta dimensión, ciertas distribuciones dificultan discriminar proximidades; además, plantillas casi idénticas pueden producir vectores poco diferenciados.

El caso de fútbol ilustra otra dificultad: incluir números en una descripción no convierte al embedding en un comparador aritmético. Por ello se utilizan filtros para restricciones y percentiles para priorizar atributos. Mejorar el índice no corrige una representación inadecuada de la necesidad del usuario.

5. Ambientes de desarrollo y aplicaciones

5.1 Alternativas de almacenamiento

La selección debe considerar infraestructura existente, control operativo, volumen, filtros e integración. Conviene distinguir motores especializados, extensiones de gestores y bibliotecas. La siguiente comparación describe su orientación, sin presentar versiones cambiantes como requisitos del prototipo.

Alternativa	Encaje en un sistema de información
Chroma	Colecciones vectoriales y APIs de documentos; usado localmente en este proyecto.
Qdrant	Motor especializado con vectores y payload; alternativa para desplegar un servicio de búsqueda.
pgvector	Extensión de PostgreSQL que integra vectores y consultas SQL con datos relacionales.
Milvus	Gestión vectorial con alternativas de despliegue orientadas a distintos volúmenes.
Weaviate	Motor con recuperación vectorial y combinación de búsqueda léxica y vectorial.
Voyager / FAISS	Bibliotecas de búsqueda; requieren resolver otras funciones de gestión en la aplicación.

Tabla 2. Alternativas y alcance general. Fuentes: documentación de los proyectos; véanse referencias.

pgvector muestra que «relacional» y «vectorial» no son categorías excluyentes. Sus límites de indexación dependen del tipo: la documentación distingue vector y halfvec, entre otros. No corresponde afirmar que todo índice HNSW tiene un límite universal de 2.000 dimensiones (pgvector, s. f.).

5.2 Lenguajes y herramientas

Python facilita la preparación de datos, los modelos y la interfaz del caso práctico. JavaScript, TypeScript, Java y otros lenguajes pueden integrarse según las APIs disponibles. LangChain, LlamaIndex y Haystack ofrecen abstracciones de orquestación; el proyecto usa módulos directos para mantener visible el recorrido desde la consulta hasta la evidencia. Las versiones efectivamente fijadas se detallan en la Sección 8.2.

5. Aplicaciones y elección de tecnología

5.3 Aplicaciones

La búsqueda semántica permite localizar contenido con descripciones aproximadas, por ejemplo procedimientos internos a partir de una pregunta. Los sistemas de recomendación pueden recuperar elementos próximos a preferencias o interacciones representadas como vectores. En detección de duplicados, la similitud permite encontrar variantes de un mismo contenido que no coinciden exactamente.

Las representaciones multimodales habilitan relaciones entre texto e imágenes cuando el modelo fue entrenado para ese espacio compartido. En código fuente se pueden buscar funciones mediante descripciones de su propósito. En soporte técnico, las incidencias históricas aportan contexto para resolver un caso nuevo. Estas aplicaciones requieren evaluar relevancia en su propio dominio.

El scouting académico desarrollado combina una necesidad conceptual con atributos estructurados. Una búsqueda por «regate y desborde» orienta la selección, mientras posición y edad pueden restringirla. El uso de RAG agrega después una explicación comparativa basada en los perfiles recuperados, si se habilita un generador.

5.4 Complementariedad con otras tecnologías

Una base relacional es adecuada para identidades, asociaciones y restricciones de integridad. Un motor de texto es útil para coincidencias léxicas y ranking por términos. El almacenamiento de objetos conserva archivos. La base vectorial ofrece acceso por similitud. Una arquitectura puede utilizar varios de estos componentes sin duplicar innecesariamente sus responsabilidades.

Si el requerimiento fuera «ordenar todos los delanteros por regate», una consulta estructurada sería una solución directa y exacta. El interés de la capa semántica aparece al interpretar descripciones variadas y recuperar perfiles afines. Esa distinción evita atribuir a la base vectorial una ventaja en tareas que no la necesitan.

Para elegir una tecnología deben estimarse volumen, frecuencia de actualización, filtros, sensibilidad de los datos y capacidad del equipo para administrarla. La prueba académica permite observar un mecanismo; la decisión productiva exige además operación, mantenimiento y evaluación con usuarios.

6. Casos de estudio

6.1 LinkedIn: información estructurada para RAG

Xu et al. (2024) describen un sistema de preguntas y respuestas de atención al cliente que utiliza grafos de conocimiento. El problema es conservar la estructura y las relaciones de incidencias históricas que se pierden cuando todo se trata como texto plano. Su propuesta recupera contexto relacionado y lo utiliza en la generación. El aporte es mostrar que organizar la fuente puede mejorar la respuesta; su traslado a otro dominio exige adaptar la construcción del grafo.

6.2 DoorDash: control de respuestas

Jia et al. (2024) presentan un sistema RAG de soporte a repartidores. Resume el problema de la conversación, recupera casos y artículos asociados, y genera una respuesta. Lo complementa con un control previo de respuestas y un evaluador de calidad. La publicación informa reducciones del 90 % en alucinaciones y del 99 % en problemas potencialmente graves de cumplimiento; son resultados declarados por sus autores en ese contexto, no métricas del presente trabajo.

El caso aporta una lección de administración: responder con una fuente no basta. Se necesitan revisión de calidad, actualización de la base y participación humana. Los componentes de control también agregan costo y latencia. La fecha de publicación del artículo es el 17 de septiembre de 2024.

6.3 Spotify: búsqueda sin generación

Voyager es una biblioteca de búsqueda aproximada desarrollada por Spotify, con interfaces para Python y Java y basada en HNSW. Permite indexar y consultar vectores sin requerir un modelo que redacte respuestas. Es una biblioteca de vecinos próximos; funciones de gestión más amplias deben resolverse en la aplicación que la integra (Spotify, s. f.).

6.4 Lectura conjunta

Los casos muestran tres responsabilidades distintas: estructurar conocimiento, controlar respuestas y recuperar vecinos. Por sí sola, la existencia de una búsqueda vectorial no implica RAG. Tampoco una mejora reportada por una organización demuestra que otro corpus obtendrá la misma calidad. El buscador de fútbol debe validarse con sus propios datos y consultas.

7. Bases vectoriales aplicadas a RAG

7.1 Recuperación y generación

Retrieval-Augmented Generation combina recuperación de información externa con generación de lenguaje. Lewis et al. (2020) distinguen la memoria paramétrica del modelo y una memoria no paramétrica consultable. La información recuperada se entrega como contexto para producir la respuesta. La arquitectura puede emplear recuperación densa, léxica u otras estrategias; no exige exclusivamente una base vectorial.

Su utilidad consiste en incorporar conocimiento específico o actualizado sin reentrenar el modelo por cada cambio. Esto no garantiza que la fuente esté vigente ni que el modelo la interprete correctamente. La actualización, el control de acceso y la selección de contenido siguen siendo responsabilidades del sistema.



Figura 2. Flujo de una consulta RAG. Elaboración propia a partir de Lewis et al. (2020).

7.2 Top-k, umbral y reordenamiento

Top-k determina cuántos resultados se conservan. Aumentarlo puede mejorar cobertura, pero también agregar contenido irrelevante y costo de contexto. Un umbral descarta resultados con puntajes insuficientes; debe calibrarse para el modelo y el dominio. Un reordenador modifica la prioridad de un conjunto recuperado, sin recuperar por sí solo los elementos que quedaron fuera.

El proyecto conserva top-k y reordenamiento por atributos, pero no implementa un umbral semántico de abstención. Si los filtros no devuelven candidatos, evita llamar al generador. Una consulta fuera del dominio puede obtener vecinos aunque sean inadecuados. La ausencia de resultados y la ausencia de evidencia relevante son situaciones diferentes.

RAG, ajuste fino y contexto extendido son combinables. El ajuste modifica parámetros; el contexto extendido entrega más material al modelo; RAG selecciona evidencia. La trazabilidad mejora cuando las referencias se verifican contra la fuente, pero una instrucción para citar no garantiza por sí sola la corrección de cada afirmación.

8. Caso práctico: buscador de jugadores

8.1 Problema, alcance y fuente

Se desarrolló una aplicación de scouting que recibe consultas en lenguaje natural y devuelve perfiles de futbolistas con información verificable en los CSV. La interfaz permite seleccionar tipo de entidad, liga, posición, edad máxima y regate mínimo, según corresponda. Muestra distancias, atributos y metadatos, y permite descargar la evidencia de la búsqueda.

El modo principal usa `male_players.csv`, `male_teams.csv` y `male_coaches.csv`. Según `manifest.json`, la selección corresponde a FIFA/EA FC edición 24, actualización 2, con fecha 22 de septiembre de 2023. Se preserva una ficha por entidad. Los modos de datos simulados y del Mundial de Clubes 2025 existen como alternativas separadas y no se mezclan con este caso.

Entidad	Cantidad
Jugadores	18.350
Equipos	702
Técnicos	1.369
Total de perfiles vectorizados	20.421

Tabla 3. Corpus del modo principal. Fuente: data/supplied/manifest.json y colección consultada.

Los archivos contienen valoraciones del videojuego y enlaces a SoFIFA. Los valores económicos tampoco representan finanzas auditadas. No se deducen contratos actuales, lesiones o rendimiento real a partir de esos campos. La fecha de los datos debe permanecer visible en toda interpretación.

Los técnicos carecen de fecha propia en su CSV y se asocian por identificador a los equipos seleccionados. Una táctica de equipo no demuestra un estilo personal del técnico ni su empleo actual. El foco de la demostración es la búsqueda de jugadores; las otras entidades muestran la posibilidad de ampliar el corpus conservando su procedencia.

El prototipo tiene alcance académico: facilita explorar perfiles y justificar candidatos dentro de los datos disponibles. No constituye un sistema validado de decisión de fichajes.

8. Caso práctico: tecnologías y arquitectura

8.2 Stack implementado

Componente	Implementación del proyecto
Interfaz	Streamlit 1.49.1; app.py y supplied_app.py.
Datos	pandas 2.2.3 y NumPy 2.2.6; CSV y manifiesto.
Embeddings	DefaultEmbeddingFunction: MiniLM local mediante ONNX, 384 dimensiones.
Base	Chroma 1.0.20; PersistentClient; HNSW y distancia coseno.
Reordenamiento	Reglas y percentiles en reranker.py; sin cross-encoder.
Generación opcional	SDK openai 1.109.1; servicio compatible u Ollama.
Configuración	python-dotenv 1.2.3; modo none sin LLM.

Tabla 4. Dependencias fijadas en requirements.txt y componentes comprobados en el código.

Chroma permite una colección persistente local sin desplegar un servidor separado. MiniLM mantiene local la vectorización y reduce la complejidad de instalación. Su ficha técnica declara 384 dimensiones y un límite de longitud de entrada; los perfiles deben ajustarse al modelo. No se presupone la misma eficacia para todo idioma o dominio (Sentence Transformers, s. f.).

La organización modular permite observar el flujo. supplied_data.py prepara registros; vector_store.py administra colección y consulta; reranker.py crea representaciones y ordena; rag_pipeline.py construye el contexto y llama opcionalmente al LLM. La comparación léxica se implementa en benchmark.py y en su adaptación al corpus.

La consulta y el perfil se vectorizan con el mismo modelo. Los documentos completos y las identidades permanecen disponibles como evidencia, aunque no todos esos campos formen parte del texto vectorizado. Activar un proveedor remoto envía la consulta y los perfiles seleccionados a ese servicio; los embeddings locales no hacen local a toda la arquitectura.

8. Caso práctico: representación y ranking

8.3 Construcción de perfiles

Para cada jugador se construye una descripción de posición, pie, fortalezas y limitaciones. Los atributos se comparan mediante percentiles dentro de grupos de campo o de arqueros. El perfil utiliza vocabulario en español e inglés para expresar capacidades; el documento detallado conserva las valoraciones exactas y la identidad. El nombre y las URL no se usan para dirigir la similitud del perfil.

Una plantilla que sólo repite atributos y cambia sus números puede generar textos demasiado similares. MiniLM no garantiza comparar «92» y «70» como lo haría una regla numérica. La representación cualitativa intenta diferenciar perfiles, mientras los filtros y el reordenador conservan el tratamiento explícito de magnitudes.

8.4 Reordenamiento por atributos

Con el reordenamiento activado, el pipeline solicita un conjunto amplio de vecinos: $\min(5.000, \max(600, k \times 1.000))$. Reconoce expresiones de la consulta y las vincula con atributos conocidos. «Regate y desborde» identifica dribbling. Si existen atributos utilizables, ordena por el promedio de sus percentiles; desempata por promedio de valoraciones originales y después por similitud.

`attribute_fit` expresa ese promedio de percentiles entre 0 y 1. No es una probabilidad ni una valoración deportiva universal. Si la consulta no contiene atributos reconocibles, o el corpus no dispone de ellos, se conserva el orden vectorial. Desactivar la opción permite examinar directamente los vecinos por distancia.

El reordenador sólo examina candidatos recuperados. Cuando el conjunto contiene todos los elegibles, el orden por atributos cubre ese universo. Si alcanza el tope y quedan jugadores fuera, un candidato valioso puede omitirse. El resultado continúa condicionado por la etapa vectorial.

Así, Chroma decide la pertenencia al conjunto candidato y el reordenador determina la prioridad por atributos. Presentar el orden final como «los cinco embeddings más similares» sería incorrecto cuando el reordenamiento está activo.

8. Caso práctico: implementación y controles

8.5 Flujo funcional



Figura 3. Arquitectura del buscador implementado. Elaboración propia a partir del código.

La primera activación del buscador prepara la colección; las siguientes reutilizan el índice si coincide el contenido. Su nombre incorpora edición, actualización y versión del perfil: `football_supplied_v24_u2_supplied_profile_v3`. La huella del corpus y la cantidad de registros permiten rechazar un índice incompatible o incompleto.

La reconstrucción es explícita. El programa no sobrescribe automáticamente una fuente modificada ni demuestra una política general de actualización por `upsert`. La metadata de finalización se guarda después de completar los lotes; una ingesta parcial no se interpreta como una colección terminada.

Los filtros de posición reconocen posiciones secundarias mediante campos `plays_ST`, `plays_CM` y equivalentes. En la interfaz se excluyen arqueros por defecto y seleccionar GK prevalece sobre esa exclusión. La consulta textual no sustituye esos controles.

8.6 Generación opcional

Con `provider=none` se muestra evidencia recuperada y no hay generación. Con un proveedor habilitado, el contexto identifica candidatos como [J1], [J2] y sucesivos. El prompt pide justificar, comparar, señalar requisitos no verificables y distinguir datos del CSV de inferencias. Es una instrucción de comportamiento, no un verificador automático de citas.

Si falla el proveedor o falta configuración, la recuperación se conserva con una advertencia. Si no existen candidatos, no se invoca al LLM. Las pruebas cubren estas rutas, pero no equivalen a demostrar que todo reporte generado será correcto.

8. Caso práctico: demostración reproducible

8.7 Procedimiento de exposición

Desde la carpeta del proyecto, iniciar la aplicación con el entorno preparado mediante el comando siguiente:

```
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Seleccionar «CSV del usuario · FIFA/EA FC» y activar el buscador local. Elegir player, posición ST, cinco resultados y el generador none. Mantener activo el reordenamiento y la comparación ANN vs LIKE. Dejar liga y edad sin restricciones, y regate mínimo en cero. Consultar: «Delantero con regate y desborde».

La misma consulta se ejecutó desde la línea de comandos. Se usó el corpus supplied, entidad player, posición ST, k=5, --provider none y --benchmark. El resultado íntegro se guardó como evidencia_busqueda.json. La búsqueda por consola conserva el filtro de género masculino y plays_ST=true; todos los recuperados corresponden al universo de delanteros de este snapshot.

8.8 Qué observar

La colección informa cantidad de vectores, dimensión y métrica. La salida muestra el filtro aplicado, los atributos detectados, el tamaño del conjunto candidato y si se agotó el universo elegible. Cada perfil incluye nombre, ID, distancia y valores que permiten verificar el resultado.

Desactivar el reordenamiento permite comparar el top-k puramente vectorial. Después puede imponerse regate mínimo 80 o una edad máxima en la interfaz y comprobar cada resultado contra el filtro. Reducir liga o posición permite estudiar la cobertura del conjunto candidato. Estas variaciones son actividades de demostración, no resultados cuantificados en la tabla de la ejecución registrada.

La recuperación local puede demostrarse sin proveedor generativo cuando el modelo y el índice ya están disponibles. La primera descarga del modelo requiere conectividad. Para exhibir RAG completo se debe activar un proveedor configurado y comprobar previamente su disponibilidad; la ejecución documentada aquí no llamó a un LLM.

8. Caso práctico: resultados comprobados

8.9 Consulta real sobre la colección completa

La ejecución del 12 de septiembre de 2026 consultó una colección de 20.421 vectores de 384 dimensiones. Los filtros seleccionaron 3.194 jugadores elegibles para ST. Chroma devolvió los 3.194 candidatos y el reordenador detectó dribbling. La cobertura fue completa para ese subconjunto, con cinco perfiles en la salida final.

N.º	Jugador	Regate	Distancia	Ajuste
1	Kylian Mbappé	92	0.5458	1.000
2	Antoine Griezmann	88	0.4895	0.999
3	Gabriel Jesus	88	0.5048	0.999
4	Christopher Nkunku	88	0.5243	0.999
5	Wissam Ben Yedder	87	0.4988	0.998

Tabla 5. Resultados reales del buscador. Distancia coseno y ajuste por percentil; valores redondeados para lectura.

Mbappé encabeza el resultado por su atributo de regate, aunque su distancia coseno es mayor que la de Griezmann. Este ejemplo hace visible que el orden final proviene del reordenamiento. Los nombres abreviados de la tabla mantienen su identificación completa en la evidencia JSON.

Medición de esta ejecución	Tiempo
Embedding de la consulta	293.80 ms
API de consulta de Chroma	2833.52 ms
Recuperación previa al reordenamiento	3142.79 ms
Escaneo léxico LIKE en pandas	7.78 ms

Tabla 6. Medición puntual local; no incluye generación de un LLM.

LIKE encontró 3.194 coincidencias con OR entre «delantero», «regate» y «desborde». Su orden es por ID, no por adecuación. En esta consulta no se demuestra superioridad de la búsqueda vectorial: la comparación es didáctica y las operaciones retornan cantidades y órdenes distintos. Los tiempos son observaciones de una ejecución, no promedios ni una prueba de escalabilidad.

8. Caso práctico: pruebas y evaluación

8.10 Verificación funcional ejecutada

Se ejecutó `python -m unittest discover -s tests -v` sobre la copia del proyecto revisada. Finalizó con 35 pruebas aprobadas, sin fallos, en 23,819 segundos. La salida se conserva en `evidencia_pruebas.txt`. Incluye controles del modo principal, el corpus alternativo y los datos de prueba; no son 35 consultas de relevancia sobre todos los jugadores.

Las pruebas verifican identidad y snapshot, asociaciones, ausencia de importes inventados, índices reales en muestras, persistencia, filtros, distancia coseno, ordenamiento y manejo de fallos. También ejercitan la interfaz con herramientas de prueba de Streamlit. Las respuestas remotas se simulan: este resultado no prueba disponibilidad ni calidad de un proveedor real.

8.11 Qué se midió y qué queda por medir

El proyecto instrumenta tiempos, cantidad de candidatos, cobertura del pool y comparación con un escaneo léxico. No contiene una evaluación etiquetada del caso principal que justifique valores de Precision@K , Recall@K o MRR . En consecuencia, no se atribuyen al buscador cifras de calidad provenientes de otro prototipo.

Para medir relevancia se propone construir consultas con criterios explícitos y jugadores considerados relevantes por evaluadores. Precision@K sería la proporción de relevantes entre k resultados; Recall@K , la fracción recuperada de todos los relevantes; MRR , el promedio del inverso de la primera posición relevante. Debe fijarse el tratamiento de consultas sin relevantes y de resultados insuficientes.

La calidad del índice es otra evaluación: comparar ANN contra vecinos exactos del mismo embedding. Obtener buen recall de vecinos no demuestra buen juicio futbolístico. La evaluación de la respuesta generada agrega fidelidad al contexto, corrección de referencias y claridad de las limitaciones.

Un banco futuro debe comparar orden vectorial, orden por atributos y un ranking estructurado sobre todos los elegibles, además de mantener filtros idénticos. También debe incluir consultas fuera del dominio, restricciones incompatibles y términos no reconocidos. Hasta ejecutar ese protocolo, las conclusiones se limitan al funcionamiento observado.

9. Análisis crítico y administración

9.1 Ventajas y limitaciones

El buscador vuelve observable la recuperación: muestra filtros, distancias, atributos y perfiles. La persistencia evita repetir la ingesta y el modo sin LLM permite inspeccionar resultados de manera independiente. El principal aprendizaje es que una solución útil puede combinar datos estructurados y similitud en etapas con responsabilidades claras.

Sus límites incluyen datos históricos de videojuego, dependencia del modelo, reconocimiento de atributos basado en reglas y posible pérdida de cobertura al limitar el pool a 5.000. No incorpora autenticación multiusuario ni una abstención calibrada por relevancia. Una recomendación dentro del corpus no demuestra que el candidato sea apropiado en el fútbol real.

9.2 Seguridad y gobierno de la información

Los embeddings no deben tratarse como datos anonimizados. La protección debe contemplar documentos, metadatos y representaciones. El catálogo OWASP de 2025 incluye debilidades de vectores y embeddings, como exposición y manipulación de información recuperada (OWASP GenAI Security Project, 2025).

El contenido recuperado se debe interpretar como datos, no como instrucciones operativas. El prompt del proyecto expresa esa separación, pero no constituye una defensa completa contra inyección de instrucciones. Un despliegue requiere controlar la ingesta y validar el uso de la salida.

El modo local conserva embeddings y colección en el equipo. Si se habilita una API externa, se transmiten la consulta y los perfiles seleccionados. Las credenciales deben quedar fuera del código, las exportaciones y los materiales de entrega. El JSON de evidencia usado aquí se obtuvo con `provider=none`.

9.3 Decisiones de gestión

La administración debe asignar responsables de fuentes, actualizaciones y recuperación ante fallos. Cada cambio de edición o modelo exige comprobar coherencia del índice y conservar capacidad de reconstrucción. Deben presupuestarse memoria, almacenamiento y, si corresponde, consumo del generador.

Para uso institucional también se requieren permisos, monitoreo, respaldo y un conjunto de evaluación mantenido. La degradación puede ser silenciosa: el sistema continúa respondiendo aunque empeoren los resultados. La calidad se debe controlar junto con disponibilidad y tiempos.

10. Conclusión

Las bases vectoriales amplían las posibilidades de recuperación de información mediante representaciones numéricas y búsqueda por similitud. Los embeddings permiten relacionar expresiones distintas y los índices aproximados reducen el trabajo de comparación a cambio de posibles omisiones. Su utilidad depende del modelo, la fuente, los filtros y la evaluación del resultado.

El desarrollo distingue esta infraestructura de RAG. La recuperación puede utilizarse sin generación, como en búsquedas y recomendaciones. Cuando se integra un LLM, la evidencia recuperada delimita el contexto disponible, pero no garantiza que la respuesta sea correcta. La trazabilidad requiere conservar fuentes y verificar que las afirmaciones realmente se apoyen en ellas.

El caso práctico implementado es un buscador de jugadores de fútbol sobre CSV de FIFA/EA FC. Utiliza Chroma persistente, MiniLM de 384 dimensiones, filtros explícitos y reordenamiento por atributos. Permite observar cómo una consulta en lenguaje natural se transforma en candidatos y cómo los valores estructurados corrigen una limitación de la representación densa para comparar magnitudes.

La consulta registrada recuperó 3.194 delanteros elegibles y seleccionó cinco perfiles ordenados por regate. El contraste entre distancia coseno y posición final mostró que ambos puntajes expresan criterios diferentes. La comparación léxica encontró coincidencias en todo el subconjunto y no demostró una ventaja general de ANN. Las 35 pruebas aprobadas aportan evidencia funcional, con el alcance específico de sus muestras y simulaciones.

No se obtuvo una medición etiquetada de relevancia ni se evaluó una respuesta generada por un proveedor real en esta revisión. Tampoco se validó el sistema como herramienta profesional de fichajes. Los datos pertenecen a una edición histórica del videojuego y deben interpretarse con esa restricción.

Las siguientes mejoras serían construir un banco de consultas evaluadas, comparar modelos adecuados al idioma y al dominio, medir recuperación exacta y aproximada y evaluar la fidelidad de las respuestas. Para pasar de demostración a servicio también harían falta controles operativos y de acceso. Desde la administración de sistemas de información, la conclusión es que estas tecnologías aportan valor cuando se integran con responsabilidades de datos, medición y mantenimiento claramente definidas.

Referencias

- Chroma. (s. f.). *Chroma documentation*. Recuperado el 12 de septiembre de 2026, de <https://docs.trychroma.com/>
- Chroma. (s. f.). *Collections*. Chroma Cookbook. Recuperado el 12 de septiembre de 2026, de <https://cookbook.chromadb.dev/core/collections/>
- Jia, Z., Wang, S., Bamba, A., & Wang, M. (2024, 17 de septiembre). *Path to high-quality LLM-based Dasher support automation*. DoorDash Engineering. <https://careersatdoordash.com/blog/large-language-modules-based-dasher-support-automation/>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474. <https://arxiv.org/abs/2005.11401>
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/>
- Milvus. (s. f.). *Milvus documentation*. <https://milvus.io/docs>
- OWASP GenAI Security Project. (2025). *LLM08:2025 Vector and embedding weaknesses*. <https://genai.owasp.org/llmrisk/llm082025-vector-and-embedding-weaknesses/>

Referencias y materiales del caso

pgvector. (s. f.). *pgvector: Open-source vector similarity search for Postgres* [Software]. GitHub. <https://github.com/pgvector/pgvector>

Qdrant. (s. f.). *Qdrant documentation*. <https://qdrant.tech/documentation/>

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. En *Proceedings of EMNLP-IJCNLP 2019* (pp. 3982-3992). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>

Sentence Transformers. (s. f.). *all-MiniLM-L6-v2* [Ficha de modelo]. Hugging Face. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

Spotify. (s. f.). *Voyager* [Software]. GitHub. <https://github.com/spotify/voyager>

Weaviate. (s. f.). *Weaviate documentation*. <https://docs.weaviate.io/weaviate>

Xu, Z., Cruz, M. J., Guevara, M., Wang, T., Deshpande, M., Wang, X., & Li, Z. (2024). Retrieval-augmented generation with knowledge graphs for customer service question answering. En *Proceedings of SIGIR 2024* (pp. 2905-2909). ACM. <https://doi.org/10.1145/3626772.3661370>

Materiales reproducibles del proyecto

Código fuente: `app.py`, `supplied_app.py`, `supplied_data.py`, `vector_store.py`, `reranker.py`, `rag_pipeline.py`, `benchmark.py` y `fifa_corpus.py`. Configuración de dependencias: `requirements.txt`. Datos: `data/supplied/male_*.csv` y `manifest.json`. Son materiales locales del proyecto revisado, sin URL pública verificada.

Evidencia adjunta: `evidencia_busqueda.json` y `evidencia_pruebas.txt`. El manifiesto conserva versión, fecha, cantidades y hashes de los CSV de origen. Los enlaces SoFIFA se preservan por registro; la autoría y licencia del conjunto original deben documentarse antes de redistribuir públicamente sus archivos.